

File in Python

Cos'è un file?

Un file è un insieme contiguo di byte usato per memorizzare dati.

Può essere semplice come un file `.txt` o complesso come un eseguibile `.exe`. Tutti i dati vengono tradotti in sequenze binarie (0 e 1) per essere processati dal computer.

I file vengono salvati su hard disk (o disco rigido): Il disco rigido è un hardware che contiene bit, una sequenza di 0 e 1 (qualsiasi cosa è una sequenza di 0 e 1 nella macchina), ma come fa il SO a capire quando finisce un file e ne inizia un altro?

Attraverso una serie di marker che caratterizzano il tipo di file:

1. Header:

contiene metadati del file, come nome, tipo, data di creazione, etc.

2. Dati:

il contenuto vero e proprio del file, cioè il corpo (o contenuto) del file.

3. EOF(end-of-file):

una sequenza di bit univoca che caratterizzano un file dall'altro, cioè è un carattere speciale che segna la fine del file.

Le estensioni dei file (`.txt`, `.jpg`, `.mp3`, etc.) indicano al sistema operativo come interpretare i byte contenuti.

Ad esempio:

- `.txt` : viene interpretato come testo (ASCII, UTF-8, etc.)
- `.jpg` : interpretato come immagine

🕒 **La deframmentazione era utile in passato per riordinare i file salvati in settori non contigui su disco (soprattutto HDD), migliorando la velocità di accesso.**

Leggere/scrivere file in Python

Quindi perché è utile poter scrivere e/o leggere sui file in Python?

In Python, i programmi vengono eseguiti in RAM, ovvero nella memoria volatile del computer. Questo significa che, una volta terminata l'esecuzione, tutte le variabili e i dati elaborati **vanno persi**, a meno che non vengano salvati in modo

permanente.

Di conseguenza questi dati vengono salvati e immagazzinati nei file così da venire scritti su disco rigido in modo che siano persistenti e permettono al programma di mantenere lo stato tra un'esecuzione e l'altra.

Altri motivi sono:

- **Scaricare la RAM:**
scrivere su file consente di liberare memoria RAM, soprattutto quando si lavora con grandi quantità di dati.
- **Configurazione e log:**
molte applicazioni leggono file di configurazione o scrivono file di log che servono per documentare eventi, errori o risultati.
- **Scambio di dati:**
i file (testuali, JSON, CSV, etc.) sono un mezzo universale per importare ed esportare informazioni tra programmi o sistemi diversi.

In sintesi: lavorare con i file in Python rende i programmi **più flessibili, scalabili e utili nel mondo reale**, dove i dati devono essere conservati e riutilizzati nel tempo.

Aprire e chiudere un file in Python

Su Python per aprire un file esiste una funzione built-in nominata `open()` : questa funzione richiede un singolo argomento; ovvero il path del file. E ritorna l'oggetto del file

La funzione `open()`:

c'è bisogno di un wrapper perché quando devo scrivere su disco rigido la scrittura del HD va ad eseguirsi tramite le API del SO quindi ho bisogno di un Wrapper.

I processi possono utilizzare risorse quindi file, quindi quando uso `open()` per aprire un file il sistema operativo lo locka agli altri utenti, per evitare sovrascrittura o cancellature dei caratteri da parte di altri utenti che magari non stanno in modalità lettura ma in modalità scrittura.

Se io blocco una risorsa quella risorsa è solo disponibile per me quindi una volta aperto devo chiudere il file senno rimarra solo disponibile per me.

Ci sono vari modi per chiudere un file in Python:



Python

```
1 reader = open("dog_breed.txt")
2 try:
```

```
3     pass
4 finally:
5     reader.close()
```

File JSON

Sono equiparabili a dei dizionari in Python:

```
1  {
2      "chiave..."{
3          "chiave": valore
4          "chiave": valore}
5  }
```

Sostanzialmente sono dizionari annidati.

Ci sono vari valori come nome_applicazioni, versione, enviroment, server, etc.

Quello che interessa è leggerli, e come si fa?

definire un Path:



Python

```
1 Path: str = "nome_directory/nome_file.json"
```

Voglio aggiungere la modalità di lettura e l'encoding:



Python

```
1 Path: str = "nome_directory/nome_file.json"
2 mode: str = "r"
3 encoding: str = "utf-8"
```

Dobbiamo anche importare una libreria (non installare con pip)



Python

```
1 import json
2 Path: str = "nome_directory/nome_file.json"
3 mode: str = "r"
4 encoding: str = "utf-8"
```

I file JSON vengono letti come testo semplici, però non sono testo semplici perché sono strutturati e Python li interpreta come dizionari.

I file JSON servono per far runnare le configurazioni di un programma, o

comunque più in generale per memorizzare informazioni.

I file Json non sono gli unici file di configurazione ma esistono anche file chiamati YAML che Python interpreta sempre come dizionari.

Apriamo il file con `open()` e una funzione della libreria json per leggere i file json:



Python

```
1 import json
2 PATH: str = "nome_directory/nome_file.json"
3 mode: str = "r"
4 encoding: str = "utf-8"
5 config = None
6
7 with open(PATH, mode=mode, encoding = encoding) as file:
8     config: dict = json.load(file)
9
10 print(config)
```

La funzione load restituisce il testo del file JSON come dizionario, quindi viene letto come testo ma restituito come un dizionario

Facendo `print(config)` restituisce tutto il testo del file JSON come un dizionario, ed è molto utile perché posso prendere il file JSON e accedere alle chiave-valori tramite l'indexing dei dizionari:



Python

```
1 import json
2 PATH: str = "nome_directory/nome_file.json"
3 mode: str = "r"
4 encoding: str = "utf-8"
5 config = None
6
7 with open(PATH, mode=mode, encoding = encoding) as file:
8     config: dict = json.load(file)
9
10 print(f'Host:{config['nome_chiave']['nome_chiave']} Port:
    {config['nome_chiave']['nome_chiave']}')
```

È buona prassi mettere le configurazioni di un programma in un file JSON a parte e importare il file JSON nel programma facendo quindi in modo che il programma prenda le configurazioni JSON e le importa nel codice del programma.

Difatti tra le varie directory del progetto è una buona practise creare una cartella

nominata `config` in cui mettere tutti i file JSON e/o YAML.

Inoltre se ho più configurazioni devo avere più file JSON inoltre per ogni configurazione devo creare più cartelle riportando il nome della configurazione e poi avere una cartella "main" dove avere un file `main.json` che prende tutte le diverse configurazioni dei diversi file posti nelle diverse sottocartelle di configurazione.

Dopodiché bisogna importare questo file `main.json` sul file di Python.

Sintassi in JSON

Io posso leggerli, usarli e modificarli e in più posso avere il mio dizionario python e sovrascriverlo e formattarlo come JSON:



Python

```
1  import json
2  PATH: str = "nome_directory/nome_file.json"
3  mode: str = "r"
4  encoding: str = "utf-8"
5  config = None
6
7  with open(PATH, mode=mode, encoding = encoding) as file:
8      config: dict = json.load(file)
9
10 config["AGGIUNTA"] = "NUOVO"
11 config["server"]["host"] = "google.it"
12 with open(PATH, mode = "w") as file:
13     # Il dump prende due cose: l'oggetto da scrivere come file
14     JSON, in questo caso è il mio dizionario.
15     #Il secondo è quello su cui vogliamo scrivere cioè il file
16     JSON
17     # Volendo c'è un terzo parametro che indenta le righe senno
18     di default mette tutto su una sola riga
19     json.dump(config, file, indent = 4)
20
21 #print(f'Host:{config['nome_chiave']['nome_chiave']} Port:
22 {config['nome_chiave']['nome_chiave']})
```

Un altro caso quello in cui si vuole creare un file JSON che non esiste partendo da un dizionario che abbiamo sul programma e lo vogliamo convertire come file JSON:



```
1 import json
2 PATH: str = "nome_directory/nome_file.json"
3 mode: str = "r"
4 encoding: str = "utf-8"
5 #config = None
6
7 with open(PATH, mode=mode, encoding = encoding) as file:
8     config: dict = json.load(file)
9
10 config: dict = {"nome_chiave": {"nome_chiave": valore,
11     "nome_chiave": "valore"}}
12 #config["AGGIUNTA"] = "NUOVO"
13 #config["server"]["host"] = "google.it"
14 with open(PATH, mode = "w") as file:
15     # Il dump prende due cose: l'oggetto da scrivere come file
16     # JSON, in questo caso è il mio dizionario.
17     #Il secondo è quello su cui vogliamo scrivere cioè il file
18     # JSON
19     # Volendo c'è un terzo parametro che indenta le righe senno
20     # di default mette tutto su una sola riga
21     json.dump(config, file, indent = 4)
22
23 #print(f'Host:{config['nome_chiave']['nome_chiave']} Port:
24     {config['nome_chiave']['nome_chiave']})
```